

430.211 Programming Methodology (프로그래밍 방법론)

Taesup Moon

Department of Electrical and Computer Engineering
Seoul National University

Outline

- Chapter 7: Constructors and Other tools
 - Constructors
 - More tools

Ch 7. Constructors and Other Tools

Learning Objectives

- Constructors
 - Definitions
 - Calling
- More Tools
 - The `const` parameter modifier
 - Inline functions
 - Static member data
 - Vector

Constructors

- Initialization of class objects
 - Initialize **some** or **all** member variables
 - Other actions possible as well
- A special kind of member function
 - ***Automatically called*** when object is declared
- Very useful tool
 - Key principle of OOP

Outline

- Chapter 7: Constructors and Other tools
 - Constructors
 - More tools

Constructor Definitions

- Constructors are defined like any member functions
 - Except that they
 1. Must have the same name as the class
 2. Cannot return a value; not even void!

```
class DayOfYear
{
public:
    DayOfYear(int monthValue, int dayValue); ← Constructor
    //Initializes the month and day to arguments.

    void input( );
    void output( );
    void set(int newMonth, int newDay);
    void set(int newMonth);
    int getMonthNumber();
    int getDay( );
private:
    int month;
    int day;
};
```

Constructor Note

- Constructor is placed in the **public section**
 - It is called when objects are declared
 - If it were in the private section, you cannot declare any objects!

Calling Constructors

- Declare objects:

```
DayOfYear date1(7, 4), date2(5, 5);
```

- Objects are created here
 - Constructor is called
 - Values in parentheses are passed as arguments to constructor
 - Member variables month, day are initialized:

date1.month	→	7	date2.month	→	5
date1.day	→	4	date2.day	→	5

Conceptual Equivalency of Constructors

- Consider:

```
DayOfYear date1, date2; //PROBLEMS—BUT FIXABLE  
date1.DayOfYear(7, 4); //VERY ILLEGAL  
date2.DayOfYear(5, 5); //VERY ILLEGAL
```

- Conceptually equivalent, but...
 - **CANNOT** call constructors like other member functions!

Constructor Definition

- Constructor definition is like all other member functions:

Class name

Constructor name

```
DayOfYear::DayOfYear(int monthValue, int dayValue)
{
    month = monthValue;
    day = dayValue;
}
```

- Note the same name around ::
 - Clearly identifies a constructor
- Again, note **no** return type
 - Just as in class definition

Alternative Definition

- Previous definition equivalent to:

```
DayOfYear::DayOfYear(int monthValue, int dayValue)  
    : month(monthValue), day(dayValue)  
{/*Body intentionally empty*/}
```

Initialization Section

- Body may be left empty

Additional Purpose of Constructors

- Not just for initializing the data
- Body doesn't have to be empty
 - **You can validate the data!** → Ensure only appropriate data is assigned to class private member variables

```
DayOfYear::DayOfYear(int monthValue, int dayValue)
    : month(monthValue), day(dayValue)
{
    if ((month < 1) || (month > 12))
    {
        cout << "Illegal month value!\n";
        exit(1);
    }
    if ((day < 1) || (day > 31))
    {
        cout << "Illegal day value!\n";
        exit(1);
    }
}
```

Overloaded Constructors

- **Can overload constructors** just like other functions
- Recall that a signature consists of:
 - Name of function
 - Parameter list
- May provide constructors for all possible argument-lists
 - Particularly "how many"

Class with Constructors Example:

Display 7.1 Class with Constructors (1 of 3)

Display 7.1 Class with Constructors (part 1 of 2)

```
1  #include <iostream>
2  #include <cstdlib> //for exit
3  using namespace std;

4  class DayOfYear
5  {
6  public:
7      DayOfYear(int monthValue, int dayValue);
8          //Initializes the month and day to arguments.

9      DayOfYear(int monthValue);
10         //Initializes the date to the first of the given month.

11     DayOfYear( ); ← default constructor
12         //Initializes the date to January 1.

13     void input( );
14     void output( );
15     int getMonthNumber( );
16     //Returns 1 for January, 2 for February, etc.
```

This definition of DayOfYear is an improved version of the class DayOfYear given in Display 6.4.

Overloaded constructors!

Class with Constructors Example:

Display 7.1 Class with Constructors (2 of 3)

```
17     int getDay( );
18 private:
19     int month;
20     int day;
21     void testDate( );
22 };
```

```
23 int main( )
24 {
25     DayOfYear date1(2, 21), date2(5), date3;
26     cout << "Initialized dates:\n";
27     date1.output( ); cout << endl;
28     date2.output( ); cout << endl;
29     date3.output( ); cout << endl;
```

This causes a call to the default constructor. Notice that there are no parentheses.

```
30     date1 = DayOfYear(10, 31);
31     cout << "date1 reset to the following:\n";
32     date1.output( ); cout << endl;
33     return 0;
34 }
```

*an explicit call to the constructor
DayOfYear::DayOfYear*

```
35
36 DayOfYear::DayOfYear(int monthValue, int dayValue)
37     : month(monthValue), day(dayValue)
38 {
39     testDate( );
40 }
```

Class with Constructors Example:

Display 7.1 Class with Constructors (3 of 3)

```
41 DayOfYear::DayOfYear(int monthValue) : month(monthValue), day(1)
42 {
43     testDate( );
44 }

45 DayOfYear::DayOfYear( ) : month(1), day(1)
46 { /*Body intentionally empty.*/}

47 //uses iostream and cstdlib:
48 void DayOfYear::testDate( )
49 {
50     if ((month < 1) || (month > 12))
51     {
52         cout << "Illegal month value!\n";
53         exit(1);
54     }
55     if ((day < 1) || (day > 31))
56     {
57         cout << "Illegal day value!\n";
58         exit(1);
59     }
60 }
```

*<Definitions of the other member
functions are the same as in
Display 6.4.>*

Sample Dialogue

```
Initialized dates:
February 21
May 1
January 1
date1 reset to the following:
October 31
```

Explicit Constructor Calls

- Can also call constructor again
 - After object declared
 - Can call via object's name; standard member function call

```
holiday = DayOfYear(5, 5);
```

- Create & initialize an anonymous object and assign it
- `Holiday=DayOfYear()` ; **(Needs the parentheses!)**

- Convenient method of setting member variables
- Method quite different from standard member function call (e.g., no return type etc.)

Default Constructors

- Constructor with no arguments
 - Make it a practice to always include the default constructor
 - If no constructors defined, a default constructor doing nothing will be automatically created
 - Otherwise (i.e., if other constructors exist), if you want to declare like `SampleClass myVar;` you need to define a default constructor

```
class SampleClass
{
public:
    SampleClass(int parameter1, double parameter2);
    SampleClass( ); ← Default constructor
    void doStuff( );
private:
    int data1;
    double data2;
};
```

```
SampleClass::SampleClass( )
{ /*Do nothing.*/ } ■
```

See Display 7.2

Class Type Member Variables

- Class member variables can be any type
 - Including objects of other classes!
 - Type of class relationship
- Need special notation for constructors
 - So they can call "back" to member object's constructor

Class Member Variable Example:

Display 7.3 A Class Member Variable (1 of 4)

Display 7.3 A Class Member Variable (part 1 of 3)

```
1  #include <iostream>
2  #include<cstdlib>
3  using namespace std;

4  class DayOfYear
5  {
6  public:
7      DayOfYear(int monthValue, int dayValue);
8      DayOfYear(int monthValue);
9      DayOfYear( );
10     void input( );
11     void output( );
12     int getMonthNumber( );
13     int getDay( );
14 private:
15     int month;
16     int day;
17     void testDate( );
18 };
```

The class DayOfYear is the same as in Display 7.1, but we have repeated all the details you need for this discussion.

Class Member Variable Example:

Display 7.3 A Class Member Variable (2 of 4)

```
19 class Holiday
20 {
21 public:
22     Holiday( ); //Initializes to January 1 with no parking enforcement
23     Holiday(int month, int day, bool theEnforcement);
24     void output( );
25 private:
26     DayOfYear date;
27     bool parkingEnforcement; //true if enforced
28 };

29 int main( )
30 {
31     Holiday h(2, 14, true);
32     cout << "Testing the class Holiday.\n";
33     h.output( );
34     return 0;
35 }

36
37 Holiday::Holiday( ) : date(1, 1), parkingEnforcement(false)
38 { /*Intentionally empty*/ }

39 Holiday::Holiday(int month, int day, bool theEnforcement)
40 : date(month, day), parkingEnforcement(theEnforcement)
41 { /*Intentionally empty*/ }
```

member variable of a class type

Invocations of constructors from the class

Class Member Variable Example:

Display 7.3 A Class Member Variable (3 of 4)

```
42 void Holiday::output( )
43 {
44     date.output( );
45     cout << endl;
46     if (parkingEnforcement)
47         cout << "Parking laws will be enforced.\n";
48     else
49         cout << "Parking laws will not be enforced.\n";
50 }

51 DayOfYear::DayOfYear(int monthValue, int dayValue)
52                     : month(monthValue), day(dayValue)
53 {
54     testDate( );
55 }

56 //uses iostream and cstdlib:
57 void DayOfYear::testDate( )
58 {
59     if ((month < 1) || (month > 12))
60     {
61         cout << "Illegal month value!\n";
62         exit(1);
63     }
64     if ((day < 1) || (day > 31))
65     {
66         cout << "Illegal day value!\n";
67         exit(1);
68     }
69 }
```

Class Member Variable Example:

Display 7.3 A Class Member Variable (4 of 4)

```
71 //Uses iostream:
72 void DayOfYear::output( )
73 {
74     switch (month)
75     {
76         case 1:
77             cout << "January "; break;
78         case 2:
79             cout << "February "; break;
80         case 3:
81             cout << "March "; break;
82             .
83             .
84             .
85         case 11:
86             cout << "November "; break;
87         case 12:
88             cout << "December "; break;
89         default:
90             cout << "Error in DayOfYear::output.";
91     }
92     cout << day;
93 }
```

The omitted lines are in Display 6.3, but they are obvious enough that you should not have to look there.

Sample Dialogue

```
Testing the class Holiday.
February 14
Parking laws will be enforced.
```

Outline

- Chapter 7: Constructors and Other tools
 - Constructors
 - More tools
 - Const
 - Inline function
 - Static members
 - Vector

Parameter Passing Methods

- Efficiency of parameter passing
 - Call-by-value
 - Requires a copy to be made
 - Negligible overhead for `int` or `double`, but can be significant for passing class objects
 - Call-by-reference
 - No need to copy the argument
 - Clear advantage for class types (that have many member variables)
- It makes sense to use call-by-reference for a class
 - Even when the function does not change the parameter!

The `const` Parameter Modifier

- Compare the following two examples
 - When the function does not change the parameters, the second one is a clear winner

```
bool isLarger(BankAccount account1, BankAccount account2)
//Returns true if the balance in account1 is greater than that
//in account2. Otherwise returns false.
{
    return(account1.getBalance( ) > account2.getBalance( ));
}
```

```
bool isLarger(const BankAccount& account1,
              const BankAccount& account2)
//Returns true if the balance in account1 is greater than that
//in account2. Otherwise, returns false.
{
    return(account1.getBalance( ) > account2.getBalance( ));
}
```

The `const` Parameter Modifier

- Can protect the argument for call-by-reference
 - Use constant parameter
 - Also called constant call-by-reference parameter
 - Place keyword `const` before type
 - Makes parameter "read-only"
 - Attempt to modify parameter results in a compiler error → automatic error checking

const Modifier for Functions

- Consider the example
 - The member function `input()` can change the member variable as well
 - What if you do not want to change the member variable by calling a member function?
- Marking a function with a `const` modifier
 - Error if the function code inadvertently changes the value

```
BankAccount myAccount;  
myAccount.input( );  
myAccount.output( );
```

```
class BankAccount  
{  
public:  
    ...  
    void output( ) const;  
    ...  
}
```

[Declaration]

```
void BankAccount::output( ) const  
{  
    ...  
}
```

[Definition]

[Pitfall] Inconsistent use of `const`

- `const` is an all-or-nothing proposition
 - If a `const` is used for a class type, then every member function that does not change the member variables should also have the `const` modifier
 - Reason: function calls within function calls

```
void welcome(const BankAccount& yourAccount)
{
    cout << "Welcome to our bank.\n"
          << "The status of your account is:\n";
    yourAccount.output();
}
```

If `output()` does not have the `const` modifier, the compiler will give an error message!

Three Uses of const

- 1) `const double pi = 3.14159;`
- 2) `void display(const Student &a);`
- 3) `class Student`
 {
 public:
 void show() const; → show() function cannot
change any member variables of the Student class
 ...
 };

– Global variable, parameter, function modifier

Inline Member Functions

- Class member function definitions
 - Typically defined separately
 - But it can be defined **IN** the class definition
 - Makes function "*in-line*"
 - Compiler inserts the definition at each location where the function is invoked → efficient for **short definitions**
- Disadvantages also exist
 - Mixing the interface and implementation
 - Inline functions can only be called in the same file as the one in which it is defined
 - Less efficient for long definitions

Example of Inline Member Function

```
class BankAccount
{
public:
    BankAccount(double balance, double rate);
    BankAccount(int dollars, int cents, double rate);
    BankAccount(int dollars, double rate);
    BankAccount( );
    void update( );
    void input( );
    void output( ) const;

    double getBalance( ) const { return (accountDollars +
                                         accountCents*0.01); }

    int getDollars( ) const { return accountDollars; }

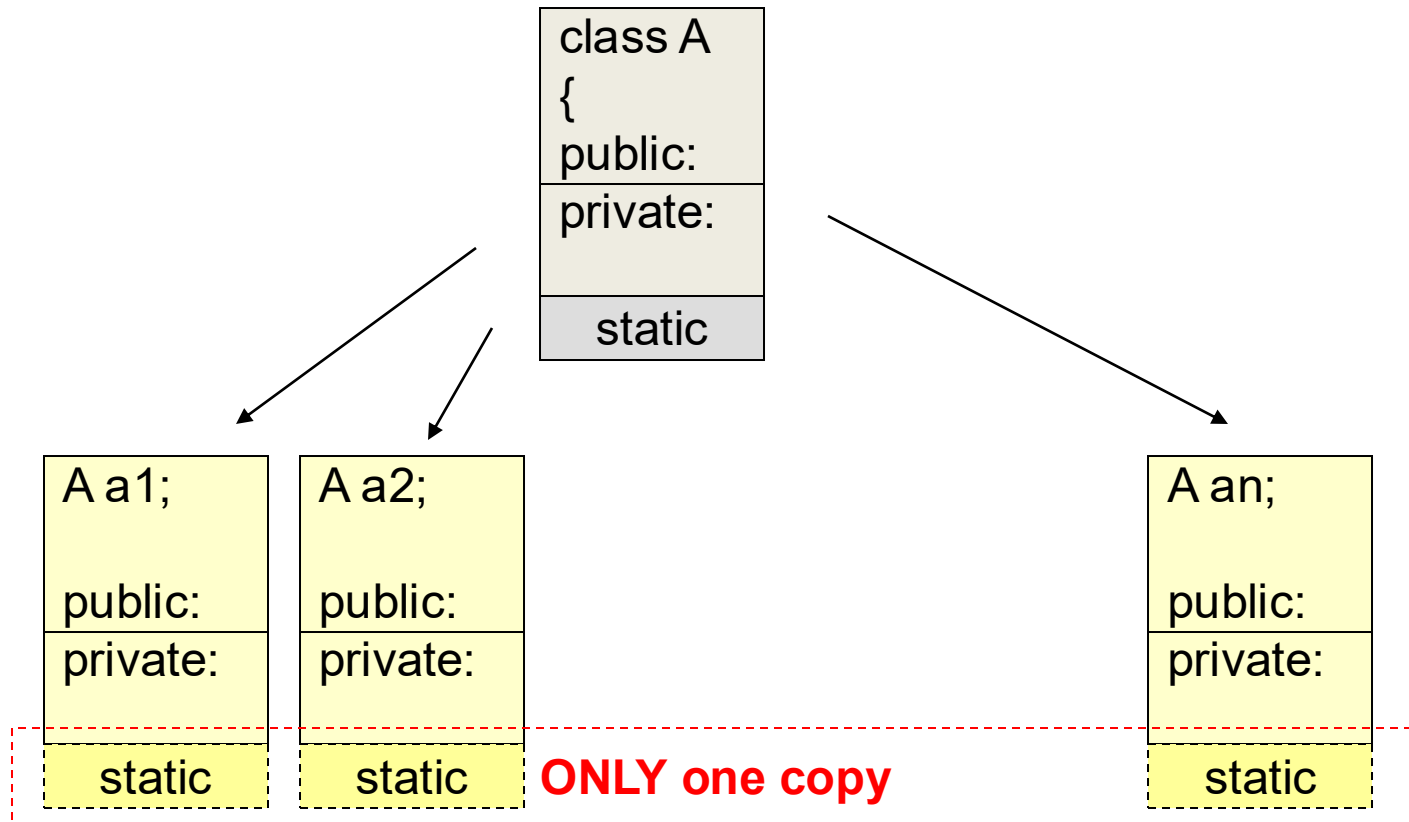
    int getCents( ) const { return accountCents; }

    double getRate( ) const { return rate; }
```

Static Members

- Static member variables
 - All objects of the class **"share"** only one copy
 - One object changes it → all can see the change
 - Similar to global variables, but only for the **objects** of the class
- Useful for "tracking"
 - How often a member function is called
 - How many objects exist at a given time
- Place keyword ***static*** before type

Static vs. Nonstatic members



Static Functions

- Class member functions can be static
 - It cannot access the object's non-static variable data
 - It can only access the static data/function of the class
 - Non-static member function can also access static variables of the class
- Then it can be called outside the class in two ways:
 - From non-class objects:
 - E.g., `Server::getTurn()` ;
 - As well as via class objects
 - Standard method: `myObject.getTurn()` ;
- Every static variable **must** be initialized outside the class definition – **only once!**

Example of Static Members

- ```
class Course
{
public:
 Course()
 {
 num_courses++;
 }
 static int getNumCourses()
 {
 return num_courses;
 }
 ...
private:
 static int num_courses;
 int num_students;
 int ID;
 ...
};

int Course::num_courses = 0;
...

```

```
int main(void)
{
 Course math, biology, physics;
 ...
 cout << "The number of courses
are: " << Course::getNumCourses();
 ...
}
```

- **Similar to global variables, but safer**

# Static Members Example:

## Display 7.6 Static Members (1 of 3)

```
1 #include <iostream>
2 using namespace std;

3 class Server
4 {
5 public:
6 Server(char letterName);
7 static int getTurn();
8 void serveOne();
9 static bool stillOpen();
10 private:
11 static int turn;
12 static int lastServed;
13 static bool nowOpen;
14 char name;
15 };
```

```
16 int Server::turn = 0;
17 int Server::lastServed = 0;
18 bool Server::nowOpen = true;
```

Initialization of static  
member variables

# Static Members Example:

## Display 7.6 Static Members (2 of 3)

```
19 int main()
20 {
21 Server s1('A'), s2('B');
22 int number, count;
23 do
24 {
25 cout << "How many in your group? ";
26 cin >> number;
27 cout << "Your turns are: ";
28 for (count = 0; count < number; count++)
29 cout << Server::getTurn() << ' ';
30 cout << endl;
31 s1.serveOne();
32 s2.serveOne();
33 } while (Server::stillOpen());
34
35 cout << "Now closing service.\n";
36
37 return 0;
38 }
```

# Static Members Example:

## Display 7.6 Static Members (3 of 3)

```
39 Server::Server(char letterName) : name(letterName)
40 { /*Intentionally empty*/}
41 int Server::getTurn() ← Since getTurn is static, only static
42 { members can be referenced in here.
43 turn++;
44 return turn;
45 }
46 bool Server::stillOpen()
47 {
48 return nowOpen;
49 }
50 void Server::serveOne()
51 {
52 if (nowOpen && lastServed < turn)
53 {
54 lastServed++;
55 cout << "Server " << name
56 << " now serving " << lastServed << endl;
57 }
58 if (lastServed >= turn) //Everyone served
59 nowOpen = false;
60 }
```

### Sample Dialogue

```
How many in your group? 3
Your turns are: 1 2 3
Server A now serving 1
Server B now serving 2
How many in your group? 2
Your turns are: 4 5
Server A now serving 3
Server B now serving 4
How many in your group? 0
Your turns are:
Server A now serving 5
Now closing service.
```

# Vectors

- Vector Introduction
  - Recall: arrays have fixed size
  - Vectors: "arrays that can grow and shrink"
    - During program execution
  - Formed from Standard Template Library (STL)
    - It uses **class template** (we will study details in a later lecture)

# Vector Basics

- Similar to array:
  - Has a base type (can also be **class types**)
  - Stores collection of base type values
- Declared differently:
  - **Syntax:** `vector<Base_Type>`
    - Indicates class template
    - Any type can be "plugged in" to Base\_Type
    - Produces a "new" class for vector with that base\_type
  - Example declaration: `vector<int> v;`

# Vector Use

- `vector<int> v;`
  - "v is vector of type int"
  - Calls a class default constructor
    - Empty vector object is created
- **Elements are indexed just like arrays for access**
  - E.g.) `v[i]=42, cout << v[i];`
  - One restriction: `v[i]` notation is used to change the value that has already been given some value
- To add new elements:
  - **Call member function** `push_back()`
- Member function `size()`
  - Returns the current number of elements

```
vector<double> sample;
sample.push_back(0.0);
sample.push_back(1.1);
sample.push_back(2.2);
```

# Vector Use

- Constructor call for vector
  - `vector<int> v(10);`
  - Will initialize the vector of size 10 (with zeros)
  - When the base type is a class type, the default constructor will be used.

# Vector Example:

## Display 7.7 Using a Vector (1 of 2)

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;

4 int main()
5 {
6 vector<int> v;
7 cout << "Enter a list of positive numbers.\n"
8 << "Place a negative number at the end.\n";

9 int next;
10 cin >> next;
11 while (next > 0)
12 {
13 v.push_back(next);
14 cout << next << " added. ";
15 cout << "v.size() = " << v.size() << endl;
16 cin >> next;
17 }
```

# Vector Example:

## Display 7.7 Using a Vector (2 of 2)

```
18 cout << "You entered:\n";
19 for (unsigned int i = 0; i < v.size(); i++)
20 cout << v[i] << " ";
21 cout << endl;

22 return 0;
23 }
```

### Sample Dialogue

```
Enter a list of positive numbers.
Place a negative number at the end.
2 4 6 8 -1
2 added. v.size = 1
4 added. v.size = 2
6 added. v.size = 3
8 added. v.size = 4
You entered:
2 4 6 8
```

# Summary I

- Constructors: automatic initialization of class data
  - Called when objects are declared
  - Constructor has same name as class
- Default constructor has no parameters
  - Should always be defined
- Class member variables
  - Can be objects of other classes
    - Require initialization-section

# Summary 2

- Constant call-by-reference parameters
  - More efficient than call-by-value
- Can *inline* very short function definitions
  - Can improve efficiency
- Static member variables
  - Shared by all objects of a class
- Vector classes
  - Like: "arrays that grow and shrink"

# Thank you!

Web: <http://mindlab-snu.github.io>

E-mail: [tsmoon@snu.ac.kr](mailto:tsmoon@snu.ac.kr)